Available online at www.sciencedirect.com
ScienceDirect
Comput. Methods Appl. Mech. Engrg. 414 (2023) 116139
Computer methods in applied mechanics and engineering
www.elsevier.com/locate/cma

Automatic boundary fitting framework of boundary dependent physics-informed neural network solving partial differential equation with complex boundary conditions

Yuchen Xie, Yu Ma, Yahui Wang*

Sino-French Institute of Nuclear Engineering and Technology, Sun Yat-sen University, Zhuhai 519082, PR China

Received 15 December 2022; received in revised form 21 May 2023; accepted 22 May 2023
Available online 21 June 2023

Abstract

The physics-informed neural network (PINN) has received much attention in the field of partial differential equation (PDE) solving due to its adaptability to different governing equations. Boundary dependent PINN (BDPINN) has much higher precision than PINN, but requires to manually design complex form of a trial function to fit boundary conditions (BCs). In this paper, an automatic boundary fitting framework is introduced into BDPINN for the automatic construction of a trial function satisfying BCs to solve PDE problems with complex geometry, and radial basis function (RBF) and neural network function (NNF) are used to construct boundary fitting functions, respectively. The proposed automatic boundary fitting framework can successfully construct a trial function of BDPINN for common BCs and their combinations, including initial conditions, Dirichlet BCs, symmetric BCs and periodic BCs. With this framework, BDPINN is applied in to 4 cases. The corresponding governing equations include diffusion, advection, and advection–diffusion equations, with dimensions ranging from 2D to 4D, and the corresponding geometric boundaries include random, irregular, and distorted boundaries. The calculation results show that the BDPINN with this framework succeeds in various governing equations, in various dimensions, and in various BCs with high accuracy, and that BDPINN with RBF is more accurate than that with NNF.

Keywords: Physics-informed neural network; Radial basis function; Partial differential equation; Deep learning

1. Introduction

Solving the partial differential equation (PDE) is a key problem for most physical problems. Most PDEs cannot be solved analytically and only approximate solutions can be obtained by numerical methods. Commonly used numerical methods are all mesh-based methods, such as finite element method, finite difference method and finite volume method. The mesh-based methods have two main disadvantages in solving PDEs: the curse of dimensionality and the discreteness of the solution. The number of mesh nodes grows exponentially with the dimension of the problem. When solving high-dimensional PDE problems, the huge mesh not only needs huge memory to store, but also makes the computation time exponentially increase. The second disadvantage is that the mesh-based method

* Corresponding author.
E-mail address: wangyh296@mail.sysu.edu.cn (Y.H. Wang).

https://doi.org/10.1016/j.cma.2023.116139
0045-7825/

can only obtain the value of the solution on the mesh point, and for the value not on the mesh points, it can only be calculated by interpolation. This disadvantage is fatal in involving multiple physics problems, because the meshes of different physics fields are not always exactly the same, and the interpolation method used in mesh mapping must introduce additional calculation errors.

Deep learning (DL), due to its powerful ability to learn and discover complex relations in large data, has attracted much attention from a number of fields, including engineering simulation [1,2], speech recognition [3,4], and image classification [5,6]. In recent years, through encoding with model equations, the physics-informed neural network (PINN) [7] has been applied to solve ordinary and partial differential equations (ODEs and PDEs) [8–12] in fluid mechanics [13–17], heat transfer [17–19], material analysis [20,21], solid mechanics [22,23] and neutron transport [24–26]. One of the advantages of PINN is its adaptability to different governing equations. With the help of automatic differentiation technology, the user only needs to give the form of the governing equation, and the training of the neural network (NN) can be completed automatically. There is no need to give corresponding discrete formats for different governing equations.

PINN approximates PDE solution by training a NN to minimize a loss function, which combines the residual of governing equation within the computational area and the deviation from boundary conditions (BCs) on the computational area's boundaries. PINN can avoid the two problems encountered by mesh-based methods in solving PDEs: curse of dimensionality and discreteness of solution. As a mesh-free method, PINN does not need to generate mesh in all computing dimensions, but randomly and discretely selects points in the entire computational area, which can avoid the exponential growth of the number of points with the number of dimensions. In addition, after the calculation, PINN gives the values of weight and bias of the trained NN, instead of the value of the solution at discrete points. Then the continuous solution defined over the entire computational area can be obtained by the well trained NN directly without interpolation.

However, in the processing of BCs, PINN has undergone many improvements. Lagaris proposed the basic idea of PINN [27], where the BCs are embedded in the trial function with an artificially designed scheme. This scheme needs to manually design the corresponding trial function form that exactly satisfies all BCs. The PINN adopts the scheme of introducing BCs is called boundary dependent physics-informed neural network (BDPINN) in this paper. For some simple BCs, this scheme is feasible. But for some more complex BCs, this scheme is extremely difficult. Therefore, Lagaris subsequently proposed two improvement schemes [28]. The first one introduces the radial basis function (RBF) in the trial function to satisfy all BCs, but the form of the trial function results in the need to recalculate the RBF at each iteration, which greatly affects the computational efficiency. The second scheme is to introduce a penalty function in the loss function to approximate all BCs. The PINN that adopts this scheme of introducing BCs is called boundary independent physics-informed neural network (BIPINN) in this paper. The deviation of the trial function from the BCs on the boundary is used as part of the loss function, and a higher penalty coefficient is given to it. The BIPINN scheme is widely used because of its simplicity and little impact on computational efficiency. However, the BIPINN processing scheme additionally introduces errors caused in fitting BCs, especially for some PDE problems that are sensitive to BCs, this error cannot be ignored. In addition, a large penalty coefficient also leads to a decrease in the fitting performance of the governing equation's residual term, thereby reducing the computational accuracy of PINN. As shown in previous work [25], the computational accuracy of BIPINN is significantly lower than that of BDPINN. In order to resolve the difficulty of BDPINN in dealing with complex Dirichlet BCs, Berg [29] introduced neural network function (NNF) as the boundary fitting function in the trial function to satisfy all BCs, and adopted a more advanced trial function form to avoid the recalculation problems. But Berg's method cannot cover various BCs that appear in practical PDE solving.

In addition to the above methods, many other PINN-like methods have also studied the processing of BCs. Different from PINN, the Deep Ritz method transforms the original problem into its weak form to reduce the smoothness requirement [30]. Based on the Deep Ritz method, Shen et al. [31] proposed penalty-Free Neural Network (PFNN) method to remove BCs from the loss term, in which, similar to Berg's work [29], an additional neural network is introduced to approximate BCs. With this approach, the Dirichlet BC can be solved. The Robin BC is processed by transforming it into Dirichlet BC form in the weak-formed governing equation. However, for more types of BCs, these works should be further developed.

Another PINN-like method is the Deep Mixed Residual Method (MIM), which uses two neural networks for trail function and its gradient separately. Meanwhile, it adds additional constraints to satisfy the gradient relationship between the two neural networks [32]. In this way, Neumann and Robin BCs are ultimately transformed into

Dirichlet BCs. However, MIM introduces an additional neural network as the gradient of the trial function, which may increase computational costs and the introduction of gradient relationship constraints may also introduce additional errors.

In this work, we propose an automatic boundary fitting framework of BDPINN for solving various PDEs with complex geometries and commonly used BCs, including Dirichlet BCs, initial conditions (ICs), symmetric BCs and periodic BCs, and their combinations. Based on the framework, the boundary fitting functions can be constructed automatically by RBF or NNF, then the trial function satisfying PDE and BCs can be constructed automatically.

Four representative cases with different governing equations, BCs and dimensions are chose to verify the feasibility of the proposed method. The rest of this paper is organized as follows. In Section 2, the principle of BDPINN, the automatic boundary fitting framework for different BCs and their combinations are introduced in order. Then, in Section 3, four representative cases with different governing equations, BCs and dimensions are chose to verify the feasibility of the proposed method. At last, main conclusions are summarized in Section 4.

2. Methodology

2.1. Governing equation and BCs

In this work, we focus on a PDE with a governing equation of form:

$$Lf = g, \forall \vec{r} \times t \in \Omega \times T = \zeta, \tag{1}$$

where L is a differential operator, f is the function to be solved, g is a forcing function, $\vec{r}$ represents the spatial variable, t represents the time variable, ζ represents the domain of definition, Ω and T are spatial and temporal domain of interest, respectively. The differential operator L can be a diffusion operator, or a convection operator, or a convection–diffusion operator. The BCs of this governing equation is summarized as:

$$Cf = h, \forall \vec{r} \times t \in \Lambda \subset \zeta, \tag{2}$$

where C is a boundary operator, h is the boundary function, Λ is the part of boundary where BCs should be imposed.

2.2. Feed-forward neural network

PINN uses a multi-layer feed-forward neural network, which is composed of several elementary functions. It passes the input variable to the output variable through several nonlinear hidden layers and can be treated as a universal function approximators [33]. A feed-forward neural network with $n+1$ layers can be described as:

$$NN(\vec{r}, t, \vec{p}) = f_a\left(f_a\left(\cdots f_a\left([\vec{r}, t][W_1] + [B_1]\right)\cdots[W_{n-1}] + [B_{n-1}]\right)[W_n] + [B_n]\right) = O, \tag{3}$$

were NN is the neural network, f_a is the activation function, $[W]$ and $[B]$ with subscript are the weight matrix and the bias matrix, respectively, $\vec{p}$ is the tuning parameters representing the ensemble of all weight and bias matrix, $[\vec{r}, t]$ is the row vector consist of $\vec{r}$ and t, as the input of NN, and O is the output variable. Common activation functions used in the deep learning include the *sigmoid* function, the hyperbolic tangent function and the ReLU function. According to early work [25], the hyperbolic tangent function has a good performance and is therefore chosen as the activation function for all four cases in this work.

2.3. BDPINN

Briefly, BDPINN assume a trial function with trial neural network (TNN), and make the trial function gradually approximate the solution of the PDE by repeatedly modifying weight and bias matrix and reevaluating the performance of the trial function on the governing equation.

A schematic diagram of BDPINN is shown in Fig. 1. The first step is the construction of TNN. The spatial and temporary variable $\vec{r}$ and t form the input variable, which is transferred through multiple hidden nonlinear layers, then the output array is obtained.

The next step is to define a trial function, which is treated as an approximation of PDE solution. In BIPINN, the trial function is directly defined by the TNN:

$$f_t(\vec{r}, t) = NN(\vec{r}, t, \vec{p}). \tag{4}$$

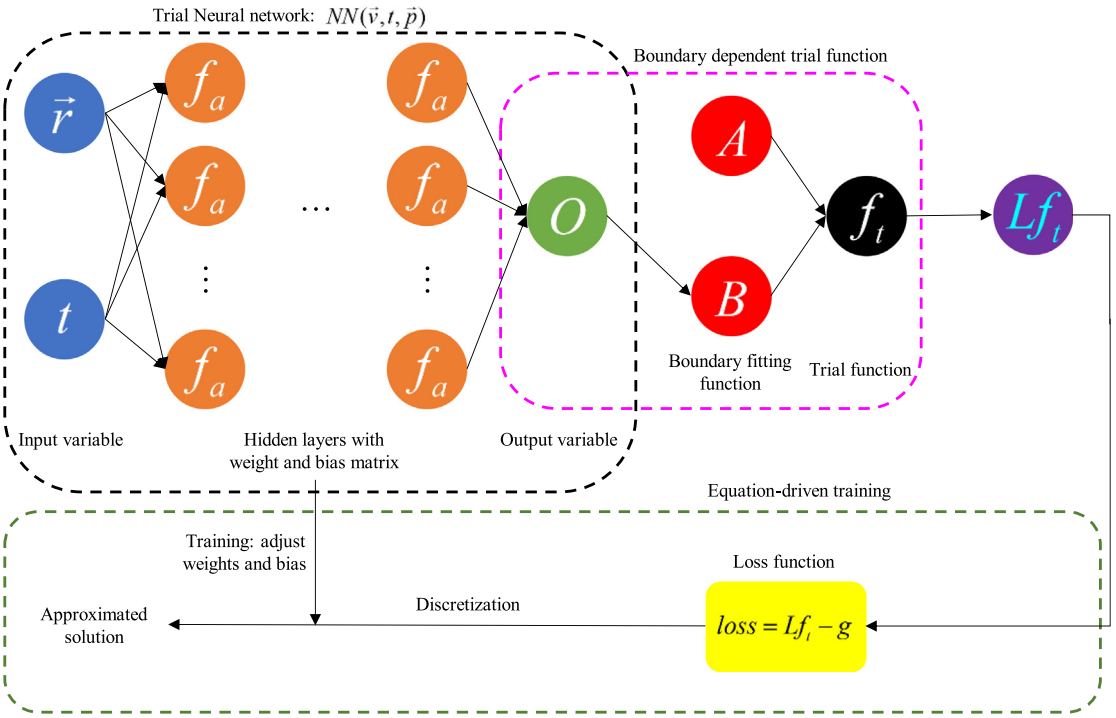

Fig. 1. Schematic diagram of BDPINN.

While in BDPINN, the trial function is improved called boundary-dependent trial function includes the constraints of BCs and is described as:

$$f_t(\vec{r}, t, \vec{p}) = A(\vec{r}, t) + B(\vec{r}, t)NN(\vec{r}, t, \vec{p}), \tag{5}$$

where $A(\vec{r}, t)$ and $B(\vec{r}, t)$ are called boundary fitting function and $NN(\vec{r}, t, \vec{p})$ is the TNN. The properties and generations of these two boundary fitting functions are detailed in Section 2.4. This trial function of BDPINN naturally satisfies all BCs and retains the universal approximation ability in the computational area.

The final step is called equation-driven training. BDPINN substitutes the boundary-dependent trial function into the governing equation and define this residual as the loss function:

$$loss(\vec{r}, t, \vec{p}) = Lf_t - g, \forall(\vec{r}, t) \in \zeta \tag{6}$$

The loss function inherits the governing equation, which means that when the loss is always equal to zero in the computational area, the boundary dependent trial function can perfectly satisfy the governing equation and can represent the solution of PDE. At this point, the PDE solving problem is transformed into a problem of finding a proper $\vec{p}$ such that $\forall(\vec{r}, t) \in \zeta, loss(\vec{r}, t, \vec{p}) = 0$.

The proper $\vec{p}$ of making $loss(\vec{r}, t, \vec{p}) = 0$ is difficult to find out. Two approximate strategies are adopted. The first one is to randomly choice a set of discrete points, which is called training set, within the continuous computational area. The second approximate strategy is to find a proper $\vec{p}$ such that $loss(\vec{r}, t, \vec{p})$ is as close to zero as possible on the training set. Through these two approximate strategies, the loss function on training set can be defined by L^2 norm:

$$loss_{train}(\vec{p}) = \sum_{i=1}^{N_{train}} (Lf_t(\vec{r}_i, t_i) - g(\vec{r}_i, t_i))^2 \tag{7}$$

where the subscript i represents the points in the training set, and N_{train} is the number of the discrete points. Then the PDE problem is finally transformed into an optimization problem of Eq. (7), with the optimizable variable $\vec{p}$.

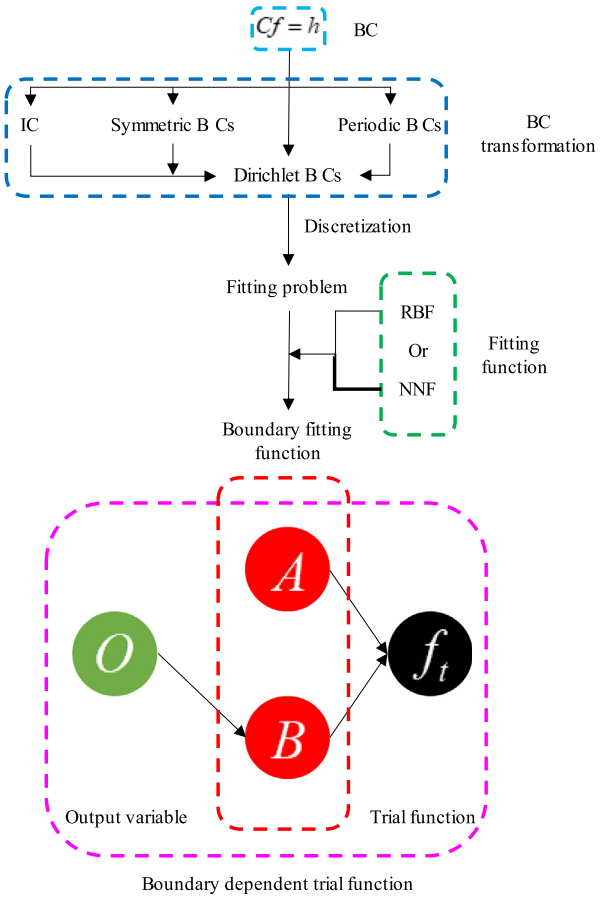

Fig. 2. Automatic boundary fitting framework.

Adjusting the tuning parameter $\vec{p}$ to make the loss function on training set as close to zero as possible, this is the meaning of training. In deep learning, the optimization of loss function is practiced by first-order gradient descent methods, such as Adam [34], or second-order gradient descent methods, such as LBFGS [35], which is chosen in this work.

2.4. Automatic boundary fitting framework

According to Section 2.3, the most challenging work of BDPINN is constructing boundary dependent trial functions. For different BCs, the trial functions need to be designed separately according to different BCs. Designing trial functions is feasible for some simple BCs, but difficult for complex BCs. In this paper, we propose a general method to automatically generate the boundary fitting functions for common BCs, then a boundary dependent trial function for common BCs can automatically be constructed.

The automatic boundary fitting framework is illustrated in Fig. 2. First, ICs, symmetric BCs and periodic BCs are converted to Dirichlet BCs. Then, through discretizing continuous boundary into a set of points, Dirichlet BCs are transformed into fitting problems, and the fitting function, RBF or NNF, can be used to solve this fitting problem to determine the boundary fitting functions A and B, which are important to construct boundary dependent trial functions.

Before presenting the generation method, the requirements for the boundary fitting functions should be firstly clarified. In this work, the boundary fitting functions are assumed to be smooth, because in most cases the solution of the PDE is smooth. In order to make sure that the boundary-dependent trial function strictly satisfies the BCs of equation reference goes here with the boundary fitting functions, we assume that the boundary fitting functions A and B in Eq. (5) satisfies the following constraint conditions:

$$CA(\vec{r},t) = h, \forall \vec{r} \times t \in \Lambda \tag{8}$$

$$B(\vec{r},t) = 0, \forall \vec{r} \times t \in \Lambda \tag{9}$$

$$B(\vec{r},t) \neq 0, \vec{r} \times t \in \zeta, \Lambda \tag{10}$$

Eqs. (8)–(10) imply that the function A itself guarantees the BCs strictly, while the function B is non-zero on the whole computational area except on the boundary Λ. The purpose of this framework is to find a smooth function A and B that satisfy Eqs. (8)–(10). For different boundary operators C, i.e., different BCs, the corresponding search methods for boundary fitting function A and B are given in the following subsubsections. Sections 2.4.1 and 2.4.2 show how to obtain A and B for Dirichlet BCs by fitting. Section 2.4.3 through Section 2.4.6 show how to convert other BCs including IC, symmetric BC and periodic BC to Dirichlet BCs.

2.4.1. Single Dirichlet BC

The base of automatic boundary fitting framework is the search of boundary fitting functions for a single Dirichlet BC, described as:

$$f(\vec{r},t) = h(\vec{r},t), \forall \vec{r} \times t \in \Lambda. \tag{11}$$

With the assumption of smoothness Eq. (8), the three requirements Eqs. (8)–(10) for boundary fitting functions become:

$$\begin{cases} A(\vec{r},t) = h(\vec{r},t), \forall \vec{r} \times t \in \Lambda \\ B(\vec{r},t) = 0, \forall \vec{r} \times t \in \Lambda \\ B(\vec{r}_c,t_c) = 1 \end{cases}. \tag{12}$$

where $(\vec{r}_c, t_c)$ is an arbitrary point inside Λ. For convenience, the geometric center of Λ is selected as $(\vec{r}_c, t_c)$ in this paper. Eq. (12) is an empirical approach to ensure that B is greater than 0 within the boundary. This approach is effective in practical calculations and appears in Sheng's work [31].

For numerical calculation, the continuous boundary Λ can be described as a subset Λ_d consist of discrete points, which is suitable for constructing boundary fitting function, especially for complex geometries. Then, Eq. (12) can be rewritten as:

$$\begin{cases} A(\vec{r}_i,t_i) = h_i, \forall \vec{r}_i \times t_i \in \Lambda_d, i = 1,\ldots,n_d \\ B(\vec{r}_i,t_i) = 0, \forall \vec{r}_i \times t_i \in \Lambda_d, i = 1,\ldots,n_d \\ B(\vec{r}_c,t_c) = 1 \end{cases}, \tag{13}$$

where n_d is the size of discrete set Λ_d. At this point, boundary fitting function A and B need to be equal to a specific value at discrete points belong to Λ_d.

The function A and B can be fitted through RBF or NNF based on the subset Λ_d. Taking the fitting process of A as an example, the RBF fitting is parameterized as:

$$A(\vec{r},t) = \sum_{i=1}^{n_d} a_i \phi(\left\| (\vec{r},t) - (\vec{r}_i,t_i) \right\|_2) = h(\vec{r},t) \tag{14}$$

where ϕ is a radial basis function such as the Gauss function used in this paper, and a_i is the coefficients of RBF, which can be obtained as follows:

$$[a] = [\phi_{ij}]^{-1}[h] \tag{15}$$

where

$$[a] = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_{n_d} \end{bmatrix}, \qquad [h] = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_{n_d} \end{bmatrix}$$

$$[\phi_{ij}] = \begin{bmatrix} \phi\left(\left\|(\vec{r}_1, t_1) - (\vec{r}_1, t_1)\right\|_2\right) & \phi\left(\left\|(\vec{r}_2, t_2) - (\vec{r}_1, t_1)\right\|_2\right) & \cdots & \phi\left(\left\|(\vec{r}_{n_d}, t_{n_d}) - (\vec{r}_1, t_1)\right\|_2\right) \\ \phi\left(\left\|(\vec{r}_1, t_1) - (\vec{r}_2, t_2)\right\|_2\right) & \phi\left(\left\|(\vec{r}_2, t_2) - (\vec{r}_2, t_2)\right\|_2\right) & \cdots & \phi\left(\left\|(\vec{r}_{n_d}, t_{n_d}) - (\vec{r}_2, t_2)\right\|_2\right) \\ \vdots & \vdots & \ddots & \vdots \\ \phi\left(\left\|(\vec{r}_1, t_1) - (\vec{r}_{n_d}, t_{n_d})\right\|_2\right) & \phi\left(\left\|(\vec{r}_2, t_2) - (\vec{r}_{n_d}, t_{n_d})\right\|_2\right) & \cdots & \phi\left(\left\|(\vec{r}_{n_d}, t_{n_d}) - (\vec{r}_{n_d}, t_{n_d})\right\|_2\right) \end{bmatrix}. \tag{16}$$

The NNF fitting is parameterized as:

$$A(\vec{r}, t) = NNF(\vec{r}, t, \vec{p}) = h(\vec{r}, t), \tag{17}$$

in which $NNF(\vec{r}, t, \vec{p})$ is a neural network function can be optimized through loss function:

$$\underset{\vec{p}}{\operatorname{argmin}} \underbrace{\sum_{i=1}^{n_d} (NNF(\vec{r}_i, t_i, \vec{p}) - h_i)^2}_{\text{loss function}} \tag{18}$$

The optimization method include Adam [34] and LBFGS [35].

In this work, either RBF or NNF is used in BDPINN for determining boundary fitting functions A and B, called BDPINN-RBF and BDPINN-NN, respectively, and their accuracy and efficiency are compared in Section 3.

With the fitting method mentioned above, boundary fitting functions A and B satisfying Eq. (12) can be determined, then a boundary-dependent trial function satisfying a single Dirichlet BC can be automatically constructed.

2.4.2. Multiple Dirichlet BCs

Assume that the computational area of the PDE contain n Dirichlet BCs, described as follows:

$$\begin{cases} f_t = h_1, \forall \vec{r} \times t \in \Lambda_1 \\ f_t = h_2, \forall \vec{r} \times t \in \Lambda_2 \\ \qquad \vdots \\ f_t = h_i, \forall \vec{r} \times t \in \Lambda_i \\ \qquad \vdots \\ f_t = h_n, \forall \vec{r} \times t \in \Lambda_n \end{cases}, \cup_{i=1}^{n} \Lambda_i = \Lambda, \tag{19}$$

where h and Λ with subscript i represent boundary functions and subsets of Λ in ith Dirichlet BCs, respectively. With the fitting method mentioned in single Dirichlet BC, A_1, B_1, …, B_i, … and B_n can be obtained as solutions of the following fitting problems, respectively,

$$A_1 = \begin{cases} h_1, \forall \vec{r} \times t \in \Lambda_1 \\ h_2, \forall \vec{r} \times t \in \Lambda_2 \\ \vdots \\ h_n, \forall \vec{r} \times t \in \Lambda_n \end{cases}, \tag{20}$$

$$\begin{cases} B_1 = 0, \forall \vec{r} \times t \in \Lambda_1 \\ B_1(\vec{r}_{1c}, t_{1c}) = 1 \end{cases}, \ldots, \begin{cases} B_i = 0, \forall \vec{r} \times t \in \Lambda_2 \\ B_i(\vec{r}_{ic}, t_{ic}) = 1 \end{cases}, \ldots, \begin{cases} B_n = 0, \forall \vec{r} \times t \in \Lambda_n \\ B_n(\vec{r}_{nc}, t_{nc}) = 1 \end{cases}$$

where $(\vec{r}_c, t_c)$ with subscript represents the center of Λ. Only function B needs to be processed in segments of different Dirichlet BCs. This is because that B needs to be equal to 1 at the center of different Dirichlet boundary

segments. If all Dirichlet BCs are merged into one Dirichlet BC, B may not be able to satisfy the requirement that B is not equal to 0 at non-boundary locations. Finally, a trial function satisfying multiple Dirichlet BCs as Eq. (19) can be automatically constructed as:

$$f_t = A_1 + NN\prod_{i=1}^{n} B_i. \tag{21}$$

2.4.3. Dirichlet BC and IC

If the computational area of the PDE contain an IC and a Dirichlet BC described as follows:

$$\begin{cases} f(\vec{r}, t=0) = f_0(\vec{r}), \forall \vec{r} \in \Omega \\ f = h, \forall \vec{r} \times t \in \Lambda \end{cases}, \tag{22}$$

where f_0 represents the initial status of f. Considering IC as a special Dirichlet BC, the fitting method mentioned in single Dirichlet BC can be applied and A_0, B_0 and B_1 can be obtained as solutions of the following fitting problems, respectively:

$$A_0 = \begin{cases} f_0, \forall \vec{r} \in \Omega \\ h, \forall \vec{r} \times t \in \Lambda \end{cases}, \begin{cases} B_0(\vec{r}, t=0) = 0, \forall \vec{r} \in \Omega \\ B_0(\vec{r}, t \neq 0) \neq 0, \forall \vec{r} \in \Omega \end{cases}, \begin{cases} B_1 = 0, \forall \vec{r} \times t \in \Lambda \\ B_1(\vec{r}_c, t_c) = 1 \end{cases}, \tag{23}$$

in which B_0 can be simply expressed as $B_0 = t$, therefore, a trial function satisfying an IC and a Dirichlet BC (22) can be automatically constructed as:

$$f_t = A_0 + tB_1NN. \tag{24}$$

2.4.4. Dirichlet BC and symmetric BC

Assume that the solution of the PDE is symmetric about the symmetry boundary S, and the PDE contains a Dirichlet BC. The PDE after and before applying symmetry are equivalent and can be described as

$$\begin{cases} Lf = g, \forall \vec{r} \times t \in \zeta_s \\ f = h, \forall \vec{r} \times t \in \Lambda_s \\ \dfrac{\partial f}{\partial \vec{n}} = 0, \forall \vec{r} \times t \in S \end{cases} \equiv \begin{cases} Lf = g, \forall \vec{r} \times t \in \zeta \\ f = h, \forall \vec{r} \times t \in \Lambda \\ f(\vec{r}, t) = f(\vec{r}_s, t_s), \forall \vec{r} \times t \in \zeta, \vec{r}_s \times t_s \in \zeta \end{cases}, \tag{25}$$

where ζ_s and Λ_s are reduced ζ and Λ after applying symmetry, $\vec{n}$ represents the normal vector pointing outside of S and $(\vec{r}_s, t_s)$ is the symmetry point of $(\vec{r}, t)$. According to the definition of symmetry, the value of f on area ζ, ζ_s should be equal to the value of f at the corresponding point on area ζ_s. So, there is an equivalence relation:

$$\begin{cases} Lf = g, \forall \vec{r} \times t \in \zeta \\ f = h, \forall \vec{r} \times t \in \Lambda \\ f(\vec{r}, t) = f(\vec{r}_s, t_s), \forall \vec{r} \times t \in \zeta, \vec{r}_s \times t_s \in \zeta \end{cases} \equiv \begin{cases} Lf = g, \forall \vec{r} \times t \in \zeta_s \\ f(\vec{r}, t) = f(\vec{r}_s, t_s), \forall \vec{r} \times t \in \zeta,\ \zeta_s \\ f = h, \forall \vec{r} \times t \in \Lambda \end{cases}, \tag{26}$$

where $f(\vec{r}, t) = f(\vec{r}_s, t_s), \forall \vec{r} \times t \in \zeta,\ \zeta_s$ can be regarded as the definition of f in area ζ, ζ_s, which does not affect the determination of the PDE solution in area ζ_s and can be ignored. A new equivalent PDE can be described as

$$\begin{cases} Lf = g, \forall \vec{r} \times t \in \zeta_s \\ f = h, \forall \vec{r} \times t \in \Lambda \end{cases}. \tag{27}$$

At this point, the symmetric BC is removed while the computational area ζ is reduced to ζ_s. The combination of symmetric BCs and a Dirichlet BC is transformed into a single Dirichlet BC, whose corresponding trial function can be automatically constructed as Section 2.4.1.

2.4.5. Dirichlet BC and periodic BC

If the computational area of the PDE contain periodic and Dirichlet BCs, the PDE problem can be described as:

$$\begin{cases} Lf = g, \forall \vec{r} \times t \in \zeta_p \\ f = h, \forall \vec{r} \times t \in \Lambda_p \\ f\left((\vec{r}, t) + \vec{J}\right) = f(\vec{r}, t), \forall \vec{r} \times t \in P \end{cases}, \tag{28}$$

where ζ_p and Λ_p are reduced ζ and Λ after applying periodicity, P is the periodic boundary and $\vec{J}$ indicates that f is periodic along it. $\vec{J}$ can be defined in both space and time, so the periodicity discussed in this work includes both spatial periodicity and temporal periodicity. Like dealing with symmetric BC, periodic BCs are tried to be eliminated and keep only Dirichlet BCs. For this purpose, a smooth transformation kernel function $K : \zeta \rightarrow K(\zeta)$ needs to be introduced, which should satisfy:

$$\begin{cases} K\left((\vec{r}, t) + \vec{J}\right) = K(\vec{r}, t), \forall \vec{r} \times t \in \zeta \\ K(\zeta) \supset \zeta_p \\ K(\Lambda) \supset \Lambda_p \end{cases}. \tag{29}$$

The role of transformation kernel function K is to map the entire calculation area ζ into the periodic calculation area ζ_p. With introducing this transformation kernel function, a new equivalent PDE can be described as:

$$\begin{cases} L(f(K(\vec{r}, t))) = g(K(\vec{r}, t)), \forall \vec{r} \times t \in \zeta \\ f(K(\vec{r}, t)) = h(K(\vec{r}, t)), \forall \vec{r} \times t \in \Lambda \end{cases}, \tag{30}$$

By replacing $K(\vec{r}, t)$ with $(\vec{r}, t)$, the PDE (30) can be rewritten as:

$$\begin{cases} L(f(\vec{r}, t)) = g(\vec{r}, t), \forall \vec{r} \times t \in K(\zeta) \\ f(\vec{r}, t) = h(\vec{r}, t), \forall \vec{r} \times t \in K(\Lambda) \end{cases}. \tag{31}$$

According to requirement (29) that K should satisfy, ζ_p and Λ_p are subsets of $K(\zeta)$ and $K(\Lambda)$, respectively. Therefore, solving PDE (30) can achieve the solution of PDE (28).

The combination of periodic BC and a Dirichlet BC is transformed into a single Dirichlet BC, whose corresponding trial function can be described as:

$$f_t = A \circ K + B \circ K \cdot NN \circ K, \tag{32}$$

where $\circ$ represent the function composition, A and B can be obtained as the solutions of the following interpolation problem:

$$\begin{cases} A \circ K = h \circ K, \forall \vec{r} \times t \in \Lambda \\ B \circ K = 0, \forall \vec{r} \times t \in \Lambda \\ B \circ K(\vec{r}_c, t_c) = 1 \end{cases} \tag{33}$$

In Lyu's work (MIM), there is also a similar way to handle periodic BCs [32]. The MIM applies a truncated Fourier series function to each variable, which can be regarded as one of specific forms of proposed work. Although the basic ideas of these two methods are similar, both are periodic mappings of the input space, their processing results are different. MIM expanded the dimensionality of the input variables, while the transformation kernel function proposed in this work did not change the dimensionality.

2.4.6. Multiple Dirichlet BCs, IC, symmetry BC, and periodic BC

If the computational area of the PDE contain n Dirichlet BCs, an IC, symmetry BCs, and a periodic BC, described as follows:

$$\begin{cases} Lf = g, \forall \vec{r} \times t \in \zeta_{ps} \\ f(\vec{r}, t = 0) = f_0(\vec{r}), \forall \vec{r} \in \Omega_{ps} \\ \dfrac{\partial f}{\partial \vec{n}} = 0, \forall \vec{r} \times t \in S \\ f(\vec{r}, t) = f\left((\vec{r}, t) + \vec{J}\right), \forall \vec{r} \times t \in P \\ f = h_i, \forall \vec{r} \times t \in \Lambda_{ps-i}, i = 1, 2, \ldots, n, \cup_{i=1}^{n} \Lambda_{ps-i} = \Lambda_{ps} \end{cases} \tag{34}$$

where ζ_{ps}, Ω_{ps}, Λ_{ps} and Λ_{ps-i} are reduced ζ, Ω, Λ and Λ_i after applying periodicity and symmetry. Applying the mirror replication and introducing transformation kernel function to remove symmetric BCs and periodic BC, then an equivalent PDE can be obtained as:

$$\begin{cases} L(f \circ K) = g \circ K, \forall \vec{r} \times t \in \zeta_s \\ f \circ K(\vec{r}, t = 0) = f_0 \circ K(\vec{r}, t = 0), \forall \vec{r} \in \Omega_s \\ f_t \circ K = h_i \circ K, \forall \vec{r} \times t \in \Lambda_i, i = 1, 2, \ldots, n \end{cases} . \tag{35}$$

According to the fitting method used in single Dirichlet BC, A_0, B_0, B_1, B_2, ... and B_n can be fitted as the following problems, respectively:

$$\begin{aligned} A_0 \circ K &= \begin{cases} f_0, \forall \vec{r} \in \Omega_s \\ h_1 \circ K, \forall \vec{r} \times t \in \Lambda_1 \\ h_2 \circ K, \forall \vec{r} \times t \in \Lambda_2 \\ \vdots \\ h_n \circ K, \forall \vec{r} \times t \in \Lambda_n \end{cases}, \\ &\begin{cases} B_0 \circ K(\vec{r}, t = 0) = 0, \forall \vec{r} \in \Omega_s \\ B_0 \circ K(\vec{r}, t \neq 0) \neq 0, \forall \vec{r} \in \Omega_s \end{cases}, \\ &\begin{cases} B_i \circ K = 0, \forall \vec{r} \times t \in \Lambda_i \\ B_i \circ K(\vec{r}_{ic}, t_{ic}) = 1 \end{cases}, i = 1, 2, \ldots, n \end{aligned} . \tag{36}$$

The second fitting problem has a simple solution $B_0 = t$, therefore, a trial function satisfying all BCs can be given as:

$$f_t = A_0 \circ K + t \prod_{i=1}^{n} B_i \circ K \cdot NN \circ K. \tag{37}$$

3. Results and discussion

There are four cases in this work to study and verify the application of BDPINN for solving PDE with complex and irregular boundaries. Case 1 investigate the application of BDPINN on two-dimensional diffusion problem with Dirichlet BC and complex geometrical shape. Case 2 justifies the use of mirror replication when dealing with symmetric BCs. In Case 3, IC and periodic BCs are introduced at the same time. In addition, as a three-dimensional transient case, the computational efficiency of Case 3 can be used to evaluate the performance of BDPINN in dealing with high-dimensional problems. In Case 4, the advection–diffusion equation is used to demonstrate the applicability of BDPINN in complex governing equations.

All these four cases are implemented on a single NVIDIA GeForce RTX 3090 GPU card, with the help of a modified open-source library *TensorDiffEq* [36].

Table 1
Constant used in Case 1.

Constant	Value
(x_c, y_c)	(0.5, 0.4)
(x_l, y_l)	(0.3, 0.45)
(x_r, y_r)	(0.7, 0.4)
r_c	0.5
r_l	0.2
r_r	0.2

3.1. 2D diffusion problem with irregular geometry

A two-dimensional diffusion problem with irregular geometry is studied in Case 1 with BDPINN. As shown in Fig. 3, the irregular boundary of Case 1 is inspired by the slice of ventricular tissue of a rabbit heart [37] and described by some discrete points. The governing equation is a stationary, scalar and linear diffusion equation given as:

$$Lf = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = g, (x, y) \in \Omega. \tag{38}$$

In Case 1, there is only one Dirichlet BC, and is given as:

$$f(x, y) = h(x, y), (x, y) \in \Gamma, \tag{39}$$

where Γ represents the external boundary shown in Fig. 3. In practical calculation, the irregular external boundary Γ is described as a set of Γ_d consisting of 114 discrete points. Therefore, the Dirichlet BC (39) is transformed as follows:

$$f(x_i, y_i) = h(x_i, y_i), (x_i, y_i) \in \Gamma_d. \tag{40}$$

Assume an analytical solution satisfying Eq. (38) is described as follows:

$$\begin{aligned} f(x, y) \quad &= \exp\left(-\frac{(x - x_c)^2 + (y - y_c)^2}{r_c^2}\right) - \exp\left(-\frac{(x - x_l)^2 + (y - y_l)^2}{r_l^2}\right) \\ &- \exp\left(-\frac{(x - x_r)^2 + (y - y_r)^2}{r_r^2}\right) \end{aligned}, \tag{41}$$

where (x_c, y_c), (x_l, y_l) and (x_r, y_r) are the centers of the regions enclosed by the external and internal boundaries, respectively, and r_c, r_l and r_r are constants. Their values are shown in Table 1. This analytical solution is used to compute the value of f on boundary set Γ_d and the mathematical expression of g, which can be described as:

$$\begin{aligned} g(x, y) = \quad &\left(\frac{4(x - x_c)^2 - 2r_c^2}{r_c^4} + \frac{4(y - y_c)^2 - 2r_c^2}{r_c^4}\right) \exp\left(-\frac{(x - x_c)^2 + (y - y_c)^2}{r_c^2}\right) + \\ &\left(\frac{4(x - x_l)^2 - 2r_l^2}{r_l^4} + \frac{4(y - y_l)^2 - 2r_l^2}{r_l^4}\right) \exp\left(-\frac{(x - x_l)^2 + (y - y_l)^2}{r_l^2}\right) + \\ &\left(\frac{4(x - x_r)^2 - 2r_r^2}{r_r^4} + \frac{4(y - y_r)^2 - 2r_r^2}{r_r^4}\right) \exp\left(-\frac{(x - x_r)^2 + (y - y_r)^2}{r_r^2}\right) \end{aligned}. \tag{42}$$

The form of trial function is described by Eq. (5).

Before calculating the trial function, the fitted boundary functions need to be prepared in advance. In this paper, the RBF and NNF are selected to fit the boundary terms A and B. The NNF used to fit A and B is a network with four hidden layers and ten neutrons per hidden layer. The fitting error and fitting time consuming of RBF and NNF on boundary fitting functions A and B are compared in Table 2. The fitting accuracy of RBF is significantly higher than that of NNF. The maximum absolute errors of RBF fit are lower than 10^{-12}, which is ten orders of magnitude lower than that of NNF. The fitting efficiency of RBF is about five times higher than that of NNF. Such an obvious

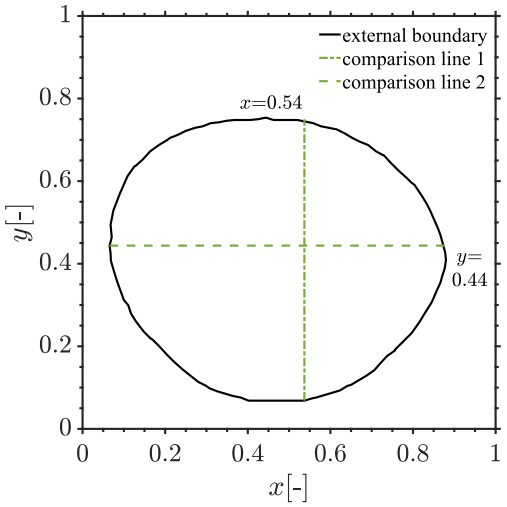

Fig. 3. Geometry of Case 1.

Table 2
Fitting performance of boundary fitting functions in Case 1.

Term	Fitting method	Absolute error		Time/s
		Maximum	Average	
A	RBF	4.45×10^{-13}	9.46×10^{-13}	5.44
	NNF	3.22×10^{-2}	6.04×10^{-3}	28.27
B	RBF	2.33×10^{-15}	7.56×10^{-16}	4.67
	NNF	9.78×10^{-3}	3.54×10^{-3}	19.44

gap between these two fitting methods is due to the difference in their fitting principles. In RBF, the fitting problem is finally transformed into a problem of solving a system of linear equations, as shown in Eq. (16), which can be solved quickly and accurately using mature and efficient linear algebra computing libraries. While in NNF, the fitting problem is finally transformed into a problem of solving a nonlinear equation, as shown in Eq. (18), for which it is usually necessary to use repeated iterations to approach the solution,

The same training set and TNN are chosen for a fair comparison of the boundary fitting functions A and B based on RBF and NNF. The size of train set is 10,000 and the TNN is a network with 12 layers and 10 neurons per hidden layer.

The global results are compared in Fig. 4. The solutions for both BDPINN-RBF and BDPINN-NNF are similar to the analytical solution. For further comparison, the results along Line 1 and Line 2 of Fig. 3 are compared in Fig. 5. The solution of BDPINN-RBF has a good agreement with the analytical solution. The accuracy and efficiency of these two methods are summarized in Table 3. BDPINN-RBF shows a marginally higher accuracy than BDPINN-NNF, but not as obvious as the advantage of accuracy on fitting function A and B, because the solution of the PDE depends on both the BCs and the governing equation. BDPINN-RBF performs much better than BDPINN-NNF in the approximation of boundary conditions, that is, the fitting of boundary functions. The computation time of BDPINN-RBF for Case 1 is a little longer than that of BDPINN-NNF, because in this case, the L-BFGS algorithm is used to train the TNN to further reduce the loss. In Case 1, BDPINN-RBF is slightly less

Table 3
Accuracy and efficiency of Case 1.

		BDPINN-RBF		BDPINN-NNF	
		Line 1	Line 2	Line 1	Line 2
Absolute error	Average	2.32×10^{-3}	2.35×10^{-3}	3.11×10^{-3}	2.83×10^{-3}
	Maximum	5.47×10^{-3}	5.88×10^{-3}	7.24×10^{-3}	2.06×10^{-2}
Time/s		2896.09		2701.62	

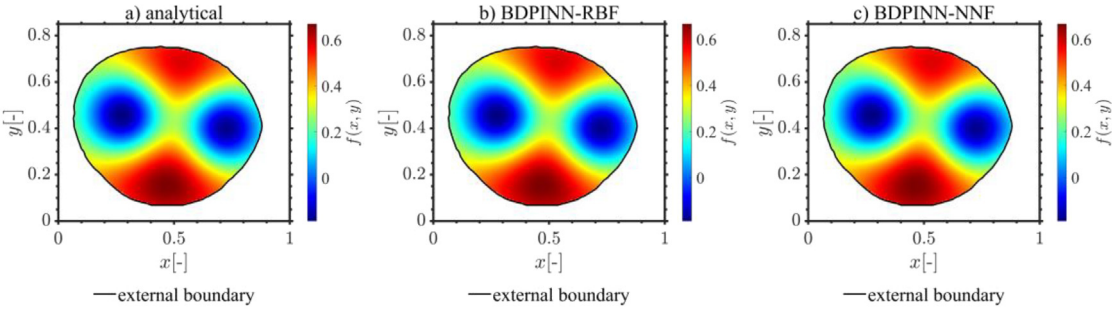

Fig. 4. Global results of Case 1.

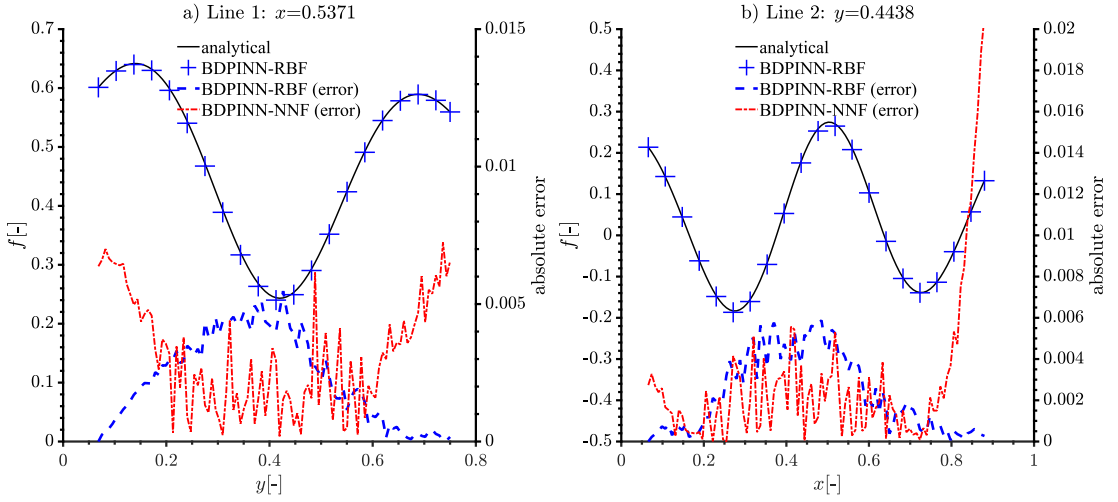

Fig. 5. Local results of Case 1 along Line 1 and Line 2.

efficient than BDPINN-NNF. This is different from the conclusions of the other three cases. This difference can be explained by statistical error.

3.2. 2D advection problem with complex edge contour

Case 2 is a two-dimensional advection problem with complex edge contour of the Koch snowflake of order four, as shown in Fig. 6. For the one-sixth Koch snowflake, the PDE problem is a stationary, scalar and linear advection

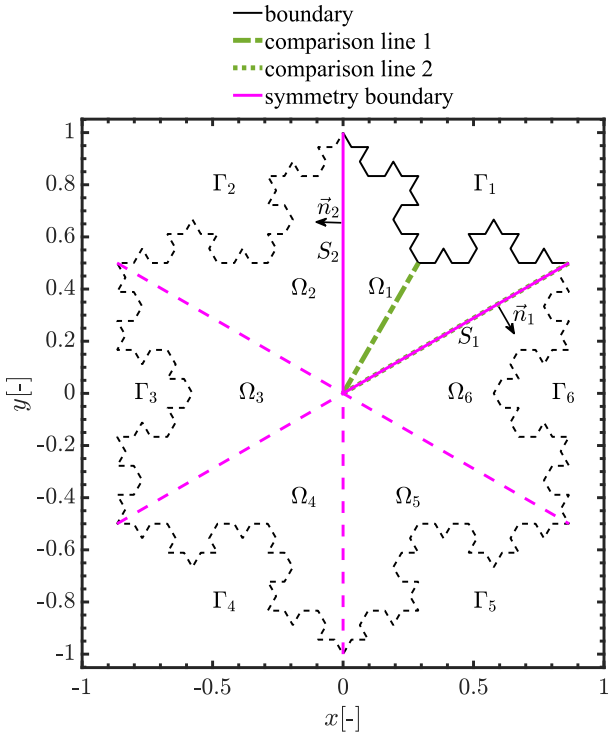

Fig. 6. Geometry of Case 2.

problem as follows:

$$\begin{cases} Lf = \dfrac{\partial f}{\partial x} + 2\dfrac{\partial f}{\partial y} = g, \forall (x, y) \in \Omega_1 \\ f(x, y) = h(x, y), (x, y) \in \Gamma_1 \\ \dfrac{\partial f}{\partial \vec{n}_i} = 0, \vec{r} \times t \in S_i, i = 1, 2 \end{cases}. \tag{43}$$

where Ω_1 represents the original computational area of one-sixth Koch snowflake surrounded by Γ_1, S_1 and S_2, Γ_1 represents Dirichlet boundary, S_1 and S_2 are two symmetry boundaries, and $\vec{n}_1$ and $\vec{n}_2$ are the outside normal vectors of S_1 and S_2, respectively. According to Section 2.4.4, the symmetric BCs are removed, and the PDE (43) is transformed to:

$$\begin{cases} Lf = \dfrac{\partial f}{\partial x} + 2\dfrac{\partial f}{\partial y} = g, \forall (x, y) \in \Omega_1 \\ f(x, y) = h(x, y), (x, y) \in \Gamma \end{cases}, \tag{44}$$

where $\Gamma = \cup \Gamma_i, i = 1, 2, \ldots, 6$ represents the boundary that surrounds the entire computational area $\Omega = \cup\ \Omega_i, i = 1, 2, \ldots, 6$, which represents the entire Koch snowflake as shown in Fig. 6. In practical calculation, Γ is discretized into the set Γ_d consisting of 192 discrete points. Therefore, the Dirichlet BC (44) is transformed to:

$$f(x_i, y_i) = h(x_i, y_i), (x_i, y_i) \in \Gamma_d. \tag{45}$$

Assume an analytical solution satisfying Eqs. (42) and (44) described as:

$$f(x, y) = \frac{1}{x^2 + y^2 + 0.1}. \tag{46}$$

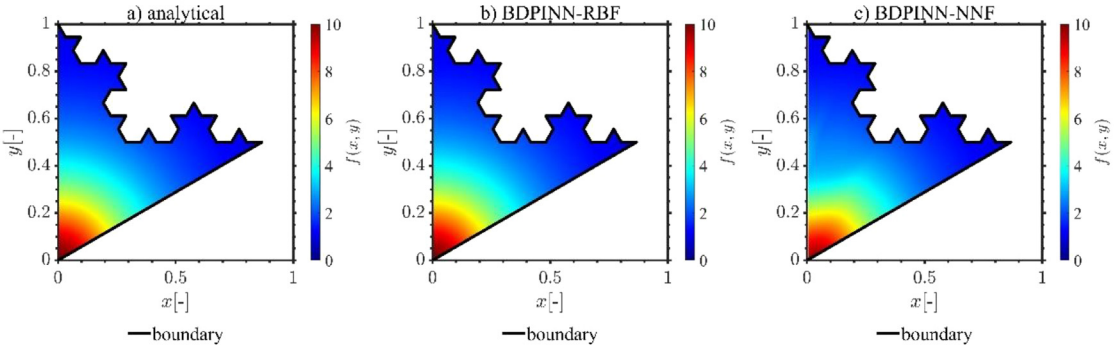

Fig. 7. Global results of Case 2.

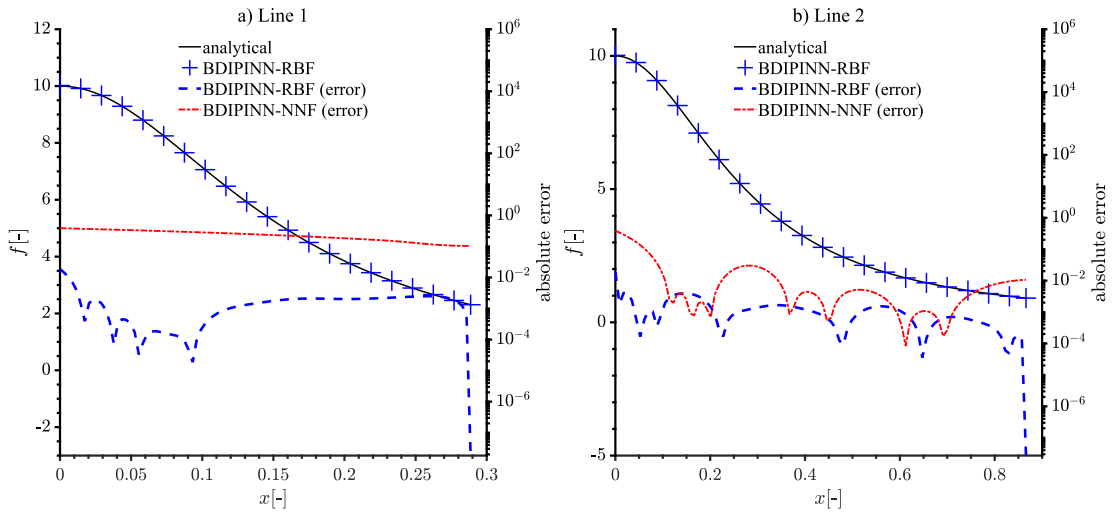

Fig. 8. Local results of Case 2 along Line 1 and Line 2.

This analytical solution is used to compute the value of h on set Γ_d and the mathematical expression of g, which can be described as:

$$g(x, y) = -\frac{200(x + 2y)}{(10x^2 + 10y^2 + 1)^2}. \tag{47}$$

The form of trial function is described by Eq. (5).

Before calculating the trial function, the boundary functions need to be prepared in advance. Both RBF and NNF are used for boundary functions fitting. The NNF used for fitting boundary functions A and B have ten and eight hidden layers, respectively, and with ten neurons in each layer. The fitting performance of RBF and NNF are compared in Table 4, which shows similar results to Case 1, i.e., both the fitting accuracy and efficiency of RBF is significantly higher than that of NNF.

The same training set and TNN structure are chosen for BDPINN-RBF and BDPINN-NNF using their corresponding boundary functions A and B. The size of train set is 10,000 and the TNN is a network with 12 layers and 12 neurons per hidden layer.

The global results are compared in Fig. 7. The solution of BDPINN-RBF has a good agreement with the analytical solution, while the solution of BDPINN-NNF is significantly different from the analytical solution. For further comparison, the results along comparison line 1 (dash-dot line) and line 2 (dotted line) of Fig. 6 are shown in Fig. 8, and the accuracy and efficiency of the two methods are summarized in Table 5. The solution of BDPINN-RBF is still close to the analytical solution, while the error of BDPINN-NNF is bigger. In Case 2, BDPINN-RBF shows a slightly higher efficiency than BDPINN-NNF, but their difference still can be explained as statistical error.

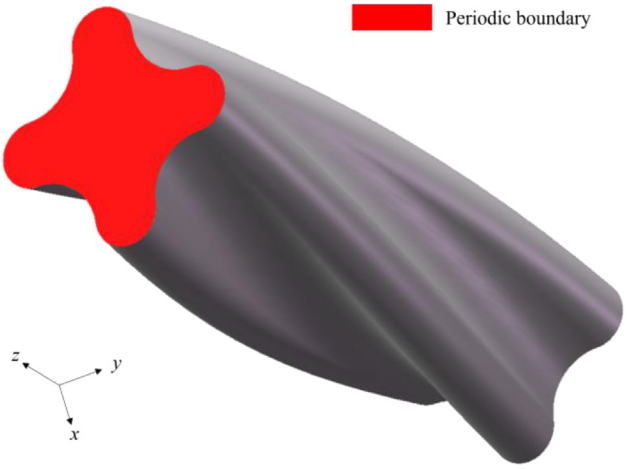

Fig. 9. Geometry of Case 3 and 4.

Table 4
Fitting performance of boundary fitting functions in Case 2.

Term	Fitting method	Absolute error		Time/s
		Maximum	Average	
A	RBF	4.15×10^{-13}	1.13×10^{-13}	4.54
	NNF	2.46×10^{-2}	7.75×10^{-3}	31.08
B	RBF	3.71×10^{-15}	9.49×10^{-16}	3.91
	NNF	1.17×10^{-2}	4.51×10^{-3}	11.70

Table 5
Accuracy and efficiency of Case 2.

		BDPINN-RBF		BDPINN-NNF	
		Line 1	Line 2	Line 1	Line 2
Absolute error	Average	1.86×10^{-3}	1.33×10^{-3}	2.40×10^{-1}	2.65×10^{-2}
	Maximum	1.87×10^{-2}	1.87×10^{-2}	3.89×10^{-1}	3.89×10^{-1}
Time/s		830.37		878.28	

3.3. 4D diffusion problem with distorted geometry

A transient three-dimensional diffusion problem with complex geometry is studied in Case 3. The geometry shape of Case 3 is periodic along z-axis with the period of one and circumferential quarter symmetry, which is inspired from the helical cruciform fuel in high-temperature reactor [38], as shown in Fig. 9. The governing equation is a time-dependent, scalar and linear diffusion equation as follows:

$$Lf = \frac{\partial f}{\partial t} + \left(\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2} \right) = g, (x, y, z, t) \in \Omega \times T, \tag{48}$$

where Ω is the geometry computational area and T is the time interval [0, 1]. Because of the periodicity and symmetry, the computational area Ω can be reduced to one quarter period Ω_p, which is surrounded by two red cross sections parallel to x–y plane (Γ_u and Γ_d) and gray screwy side boundary Ξ, as shown in Fig. 9. The governing equation is rewritten as:

$$Lf = \frac{\partial f}{\partial t} + \left(\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}\right) = g, (x, y, z, t) \in \Omega_p \times T. \tag{49}$$

The BC combination of Case 3 consists of three BCs, including IC, Dirichlet BC and periodic BC, which are described as follows,

$$f(x, y, z, 0) = f_0(x, y, z), \forall (x, y, z) \in \Omega_p, \tag{50}$$

$$f(x, y, z, t) = h(x, y, z, t), \forall (x, y, z, t) \in \Xi \times T, \tag{51}$$

$$f(x, y, 0, t) = f(x, y, 1, t), \forall (x, y, t) \in (\Omega_u \cup \Omega_d) \times T, \tag{52}$$

where f_0 represent the initial status of f. After application of the kernel transfer function K, the periodic BC disappear, leaving only Dirichlet BC and IC. In practical calculation, $\Xi \times T$ is discretized into the set $(\Xi \times T)_d$ consisting of 3040 discrete spatial points with different time coordinates. Discrete occurs simultaneously in time and space. Ω_p is discretized into the set Ω_{pd} consisting of 500 discrete spatial points. Assume an analytical solution satisfying Eqs. (49)–(52) described as

$$f(x, y, z, t) = \frac{\cos(2\pi z)}{x^2 + y^2 + 1}(t + 1). \tag{53}$$

This analytical solution is used to compute the value of h on set $(\Xi \times T)_d$ and the mathematical expression of f_0 and g, which can be described as follows:

$$f_0(x, y, z) = \frac{\cos(2\pi z)}{x^2 + y^2 + 1}, \tag{54}$$

$$\begin{aligned} g(x, y, z, t) \quad &= \cos(2z\pi)/(x^2 + y^2 + 1) \\ &-(4\cos(2z\pi)(t + 1))/(x^2 + y^2 + 1)^2 \\ &+(8x^2\cos(2z\pi)(t + 1))/(x^2 + y^2 + 1)^3. \\ &+(8y^2\cos(2z\pi)(t + 1))/(x^2 + y^2 + 1)^3 \\ &-(4\pi^2\cos(2z\pi)(t + 1))/(x^2 + y^2 + 1) \end{aligned} \tag{55}$$

The form of trial function is described by Eq. (37), where its transformation kernel function K for Case 3 is defined by:

$$K(x, y, z, t) = (x, y, \cos(2\pi z), t) \tag{56}$$

Before calculating the trial function, the fitted boundary functions need to be prepared in advance. Both RBF and NNF are used for boundary functions fitting. The NNF used for fitting have ten hidden layers with ten neurons in each layer. The fitting performance of RBF and NNF on boundary fitting function A and B are compared in Table 6. Both the fitting accuracy and efficiency of RBF is significantly higher than that of NNF.

The same training set and TNN structure are chosen for BDPINN-RBF and BDPINN-NNF using their corresponding boundary functions A and B. The size of train set is 100 and the TNN is a network with 12 layers and 12 neurons per hidden layer. The local results in cross sections are compared in Fig. 10. The solutions for both BDPINN-RBF and BDPINN-NNF are similar to the analytical solution. For further comparison, three lines are selected and their results are compared in Fig. 11. Line 1 is the straight-line $y = x$ on the section of $z = 0.5$ when $t = 0.5$. Line 2 is the axle wire when $t = 0.5$. Line 3 represents the change of point (0, 0, 0.5) over time. In conclusion, the solution of BDPINN-RBF has a good agreement with the analytical solution. The accuracy and efficiency of these two methods are summarized in Table 7, from which it can be seen that BDPINN-RBF has a better performance in both accuracy and efficiency.

It is worth noting that in Fig. 11(c), the error is close to zero when t is equal to 0, but gets bigger when t increases. This is caused by the characteristics of the transient equation. The IC accurately describe the initial state

Table 6
Fitting performance of boundary fitting functions in Case 3.

Term	Fitting method	Absolute error		Time/s
		Maximum	Average	
A	RBF	1.26×10^{-13}	2.64×10^{-14}	5.65
	NNF	3.09×10^{-2}	8.02×10^{-3}	17.70
B	RBF	1.26×10^{-9}	6.61×10^{-10}	4.90
	NNF	2.06×10^{-1}	8.67×10^{-2}	15.78

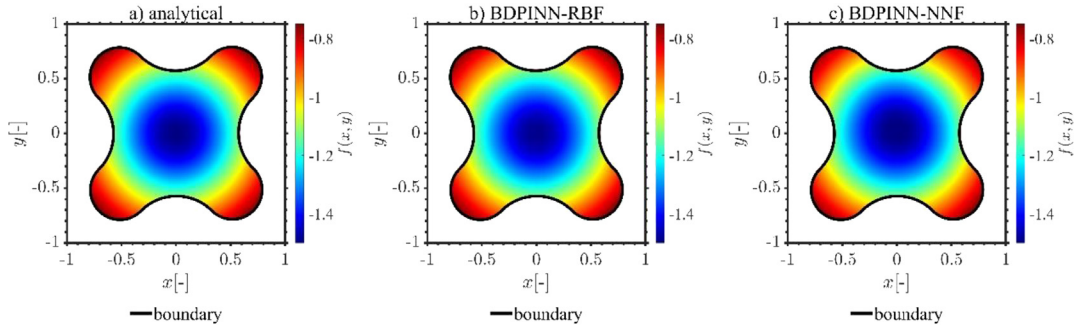

Fig. 10. Local results in section $z = 0.50, t = 0.5$ of Case 3.

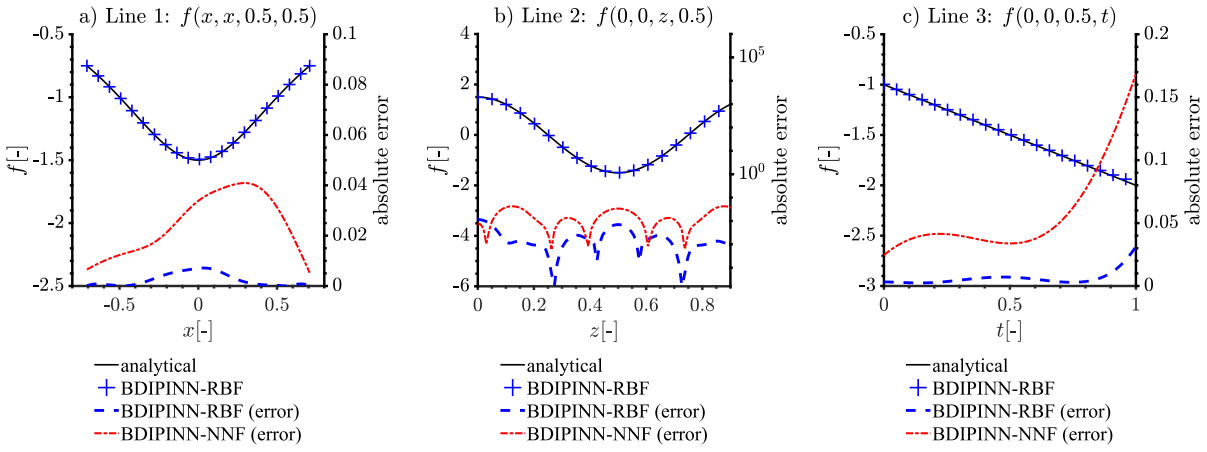

Fig. 11. Local results along Line 1, 2 and 3 of Case 3.

of the system which can be perfectly fitted by boundary functions, so the error at $t = 0$ is very small. However, the evolution of the system is determined by the governing equation. The fitting of the governing equation has a non-negligible error that will accumulate over time. Therefore, when $t = 1$, the error is the largest.

Table 7
Accuracy and efficiency of Case 3.

		BDPINN-RBF			BDPINN-NNF		
		Line 1	Line 2	Line 3	Line 1	Line 2	Line 3
Absolute error	Average	2.48×10^{-3}	2.80×10^{-3}	6.21×10^{-3}	2.48×10^{-2}	1.67×10^{-2}	5.66×10^{-2}
	Maximum	7.15×10^{-3}	1.15×10^{-2}	3.13×10^{-2}	4.09×10^{-2}	4.24×10^{-2}	1.69×10^{-1}
Time/s		158.99			195.57		

3.4. 3D advection–diffusion problem with distorted geometry

In Case 4, a three-dimensional advection–diffusion problem with the same geometry and boundary BCs as Case 3 is considered. The governing equation is a scalar and linear advection–diffusion equation and is given as:

$$Lf = -\left(\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}\right) + \left(\frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} + \frac{\partial f}{\partial z}\right) = g, (x, y, z) \in \Omega, \tag{57}$$

where Ω is the geometry boundary. Because of the periodicity and symmetry, the computational area Ω can be reduced to one quarter period Ω_p,

$$Lf = -\left(\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}\right) + \left(\frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} + \frac{\partial f}{\partial z}\right) = g, (x, y, z) \in \Omega_p. \tag{58}$$

The Dirichlet BC and periodic BC of Case 4 are described by:

$$f(x, y, z) = h(x, y, z), \forall (x, y, z) \in \Xi, \tag{59}$$

$$f(x, y, 0) = f(x, y, 1), \forall (x, y) \in \Omega_u \cup \Omega_d. \tag{60}$$

The periodic BC can be removed by applying the kernel transfer function K. In practical calculation, Ξ is discretized into the set Ξ_d consisting of 3040 discrete spatial points. Assume an analytical solution described as

$$f(x, y, z, t) = \frac{\cos(2\pi z)}{x^2 + y^2 + 1}. \tag{61}$$

This analytical solution is used to generate the value of h on set Ξ_d and the mathematical expression of g, which can be described as:

$$g = \begin{array}{l} \frac{4\cos(2z\pi)}{(x^2+y^2+1)^2} + \frac{4\pi^2\cos(2z\pi)}{x^2+y^2+1} - \\ \frac{8x^2\cos(2z\pi)}{(x^2+y^2+1)^3} - \frac{8y^2\cos(2z\pi)}{(x^2+y^2+1)^3} - \\ \frac{2\pi sin(2z\pi)}{x^2+y^2+1} - \frac{2x\cos(2z\pi)}{(x^2+y^2+1)^2} - \\ \frac{2y\cos(2z\pi)}{(x^2+y^2+1)^2} \end{array}. \tag{62}$$

The form of trial function is described by (32), where its transformation kernel function is defined as:

$$K(x, y, z) = (x, y, \cos(2\pi z)). \tag{63}$$

Before determining the trial function, both RBF and NNF are used for boundary functions fitting. The NNF used to fit boundary fitting functions A and B have twelve hidden layers with twelve neurons in each layer. The fitting performance of RBF and NNF are compared in Table 8. As in Case 3, the fitting accuracy of RBF is much better than that of NNF.

Table 8
Fitting performance of boundary fitting functions in Case 4.

Term	Fitting method	Absolute error		Time/s
		Maximum	Average	
A	RBF	3.06×10^{-10}	3.35×10^{-11}	4.23
	NNF	7.89×10^{-3}	1.64×10^{-3}	65.20
B	RBF	1.26×10^{-9}	6.61×10^{-10}	0.25
	NNF	6.08×10^{-3}	1.86×10^{-3}	56.82

Table 9
Accuracy and efficiency of Case 4.

		BDPINN-RBF		BDPINN-NNF	
		Line 1	Line 2	Line 1	Line 2
Absolute error	Average	5.27×10^{-7}	4.19×10^{-6}	2.20×10^{-2}	3.59×10^{-2}
	Maximum	1.96×10^{-6}	1.04×10^{-5}	5.38×10^{-2}	5.62×10^{-2}
Time/s		127.99		164.17	

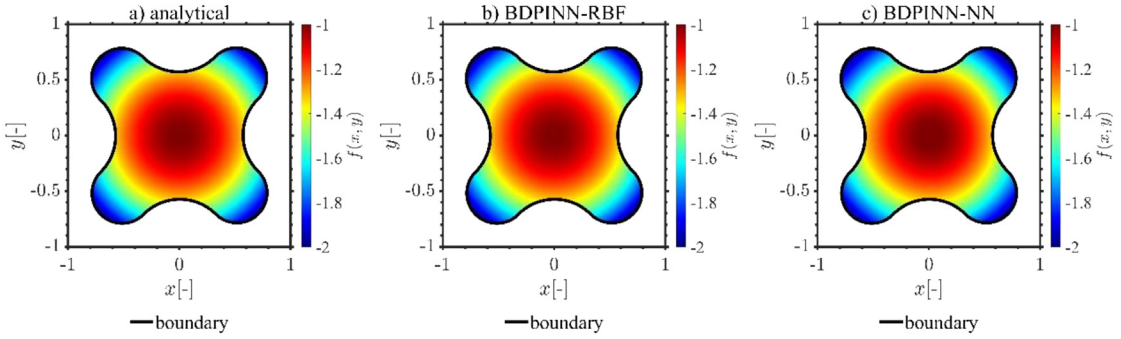

Fig. 12. Local results in section $z = 0.50$ of Case 4.

The maximum and average of absolute error of RBF on boundary fitting function B are the same as those in Case 3, because in these two cases the data of fitting function B are completely consistent. The fitting process of RBF is deterministic, which is different from NNF fitting with uncertain initial weights and biases. This also explains why in Case 4, the time cost of BRF in fitting function B is very short. A brief conclusion can be drawn from Table 8: RBF is more accurate and efficient than NNF in fitting boundary fitting functions.

The same training set and TNN are chosen as in Case 3. The local results in cross section are compared in Fig. 12. The solutions of BDPINN-RBF and BDPINN-NNF are both similar to the analytical solution. For further comparison, the results along diagonal line of $z = 0.5$ cross section (Line 1) and z-axis (Line 2) are shown in Fig. 13. The BDPINN-RBF has a much better performance than BDPINN-NNF in both accuracy and efficiency, as shown in Table 9.

4. Conclusions

Compared to BIPINN, BDPINN has higher computational accuracy, but its application range is limited to PDEs with simple BCs. In order to improve the applicability of BDPINN to PDEs with complex BCs, this paper proposes

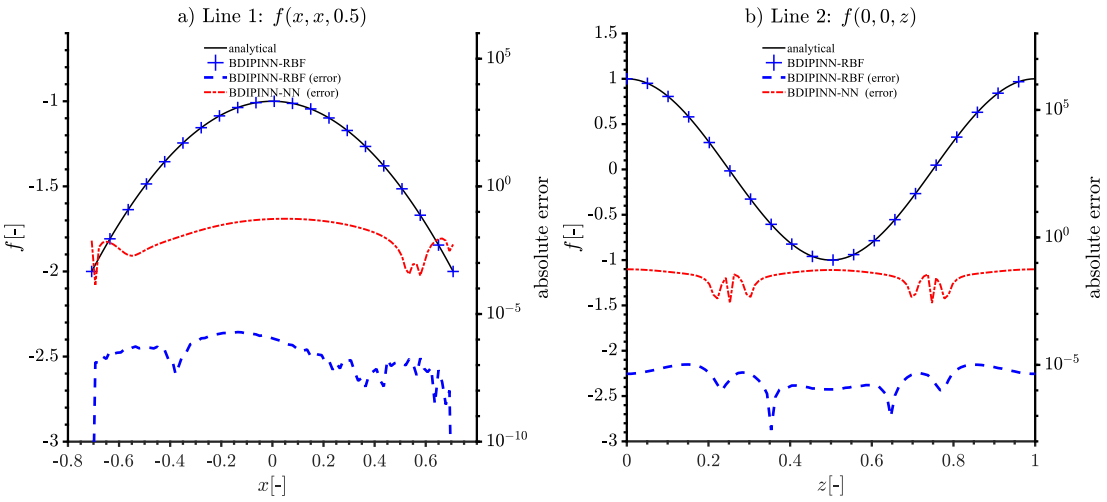

Fig. 13. Local results along Line 1 and 2 of Case 4.

an automatic calculation framework, which uses RBF or NNF to determine boundary-dependent trial function, then train TNN to describe the solution. The automatic calculation framework is successfully applied to cases with complex governing equation, geometries and BCs, and agree well with the analytical solution. In addition, compared with BDPINN-NNF, BDPINN-RBF not only has significantly higher accuracy and efficiency in the fitting of boundary fitting functions, but also shows smaller errors in TNN training.

This paper systematically organizes and presents the automatic calculation framework of BDPINN, and improves BDPINN for higher accuracy and efficiency. It is a solid foundation for the application and promotion of BDPINN. At the same time, it also provides a new perspective for PDE solving with complex BCs.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All necessary data has been added to the manuscript.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant No. 12205389), the Natural Science Foundation of Guangdong Province under (Grant No. 2022A1515011735), and Science and Technology on Reactor System Design Technology Laboratory (Grant No. KFKT-05-FW-HT-20220001).

References

[1] T.K. Kim, et al., Deep-learning-based alarm system for accident diagnosis and reactor state classification with probability value, Ann. Nucl. Energy 133 (2019) 723–731.

[2] H. Bao, et al., Using deep learning to explore local physical similarity for global-scale bridging in thermal-hydraulic simulation, Ann. Nucl. Energy 147 (2020) 107684.

[3] G. Hinton, et al., Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups, IEEE Signal Process. Mag. 29 (6) (2012) 82–97.

[4] T.N. Sainath, et al., Improvements to deep convolutional neural networks for LVCSR, in: 2013 IEEE Workshop on Automatic Speech Recognition and Understanding, IEEE, 2013.

[5] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, Adv. Neural Inf. Process. Syst. (2012) 25.

[6] J.J. Tompson, et al., Joint training of a convolutional network and a graphical model for human pose estimation, Adv. Neural Inform. Process. Syst. (2014) 27.
[7] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[8] V. Dwivedi, N. Parashar, B. Srinivasan, Distributed learning machines for solving forward and inverse problems in partial differential equations, Neurocomputing 420 (2021) 299–316.
[9] V. Dwivedi, B. Srinivasan, Physics informed extreme learning machine (pielm)–A rapid method for the numerical solution of partial differential equations, Neurocomputing 391 (2020) 96–118.
[10] Z. Xiang, et al., Self-adaptive loss balanced physics-informed neural networks, Neurocomputing 496 (2022) 11–34.
[11] Y. Ye, et al., Deep neural network methods for solving forward and inverse problems of time fractional diffusion equations with conformable derivative, Neurocomputing 509 (2022) 177–192.
[12] W. Zhong, H. Meidani, PI-VAE: Physics-informed variational auto-encoder for stochastic differential equations, Comput. Methods Appl. Mech. Engrg. 403 (2023) 115664.
[13] L. Sun, et al., Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data, Comput. Methods Appl. Mech. Eng. 361 (2020) 112732.
[14] X. Jin, et al., NSFnets (Navier–Stokes flow nets): Physics-informed neural networks for the incompressible Navier–Stokes equations, J. Comput. Phys. 426 (2021) 109951.
[15] Z. Mao, A.D. Jagtap, G.E. Karniadakis, Physics-informed neural networks for high-speed flows, Comput. Methods Appl. Mech. Engrg. 360 (2020) 112789.
[16] Q. Lou, X. Meng, G.E. Karniadakis, Physics-informed neural networks for solving forward and inverse flow problems via the Boltzmann-BGK formulation, J. Comput. Phys. 447 (2021) 110676.
[17] H. Gao, L. Sun, J.-X. Wang, PhyGeoNet: Physics-informed geometry-adaptive convolutional neural networks for solving parameterized steady-state PDEs on irregular domain, J. Comput. Phys. 428 (2021) 110079.
[18] S. Mishra, R. Molinaro, Physics informed neural networks for simulating radiative transfer, J. Quant. Spectrosc. Radiat. Transfer 270 (2021) 107705.
[19] F. Mostajeran, R. Mokhtari, DeepBHCP: Deep neural network algorithm for solving backward heat conduction problems, Comput. Phys. Comm. 272 (2022) 108236.
[20] Z. Fang, J. Zhan, Deep physical informed neural networks for metamaterial design, IEEE Access 8 (2019) 24506–24513.
[21] D. Liu, Y. Wang, Multi-fidelity physics-constrained neural network and its application in materials modeling, J. Mech. Des. 141 (12) (2019).
[22] E. Haghighat, et al., A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics, Comput. Methods Appl. Mech. Eng. 379 (2021) 113741.
[23] P. Pantidis, M.E. Mobasher, Integrated Finite Element Neural Network (I-FENN) for non-local continuum damage mechanics, Comput. Methods Appl. Mech. Engrg. 404 (2023) 115766.
[24] J. Wang, et al., Surrogate modeling for neutron diffusion problems based on conservative physics-informed neural networks with boundary conditions enforcement, Ann. Nucl. Energy 176 (2022) 109234.
[25] Y. Xie, et al., Neural network based deep learning method for multi-dimensional neutron diffusion problems with novel treatment to boundary, J. Nucl. Eng. 2 (4) (2021) 533–552.
[26] M.H. Elhareef, Z. Wu, Physics-informed neural network method and application to nuclear reactor calculations: A pilot study, Nucl. Sci. Eng. (2022) 1–22.
[27] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (5) (1998) 987–1000.
[28] I.E. Lagaris, A.C. Likas, D.G. Papageorgiou, Neural-network methods for boundary value problems with irregular boundaries, IEEE Trans. Neural Netw. 11 (5) (2000) 1041–1049.
[29] J. Berg, K. Nystrom, A unified deep artificial neural network approach to partial differential equations in complex geometries, Neurocomputing 317 (2018) 28–41.
[30] B. Yu, The deep Ritz method: A deep learning-based numerical algorithm for solving variational problems, Commun. Math. Stat. 6 (1) (2018) 1–12.
[31] H. Sheng, C. Yang, PFNN: A penalty-free neural network method for solving a class of second-order boundary-value problems on complex geometries, J. Comput. Phys. 428 (2021) 110085.
[32] L. Lyu, et al., Enforcing exact boundary and initial conditions in the deep mixed residual method, 2020, arXiv preprint arXiv:2008.01491.
[33] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, Neural Netw. 2 (5) (1989) 359–366.
[34] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.
[35] D.C. Liu, J. Nocedal, On the limited memory BFGS method for large scale optimization, Math. Program. 45 (1) (1989) 503–528.
[36] L.D. McClenny, M.A. Haile, U.M. Braga-Neto, TensorDiffEq: Scalable multi-GPU forward and inverse solvers for physics informed neural networks, 2021, arXiv preprint arXiv:2103.16034.
[37] A. Bueno-Orovio, V.M. Pérez-García, Spectral smoothed boundary methods: The role of external boundary conditions, Numer. Methods Partial Differential Equations: Int. J. 22 (2) (2006) 435–448.
[38] T. Conboy, T. McKrell, M. Kazimi, Experimental investigation of hydraulics and lateral mixing for helical-cruciform fuel rod assemblies, Nucl. Technol. 182 (3) (2013) 259–273.